

Программирование на С

Лекция 14

Многофайловые программы и препроцессор

Одна программа — один файл

Пока все наши программы выглядели примерно так.

```
main.c
```

```
|— struct Vector3  
|— add()  
|— subtract()  
|— dot()  
|— cross()  
|— length()  
|— normalize()  
|— printVector()  
└— main()
```

Что вам не нравится?

Представьте, что программа продолжает расти.

main.c

↓

500 строк

↓

1500 строк

↓

5000 строк

Попробуем разделить программу

Функции для работы с векторами можно вынести отдельно.

```
main.c
```

```
↓
```

```
main()
```

Кажется, проблема решена.

```
vector3.c
```

```
↓
```

```
add()  
subtract()  
dot()  
cross()  
length()  
normalize()  
printVector()
```

Но появилась новая проблема

В файле `main.c` используется функция

```
struct Vector3 sum = add(a, b);
```

? Откуда компилятор узнает,
что функция `add()` существует?

И ещё одна проблема

В файле `vector3.c` находится

```
struct Vector3 add(struct Vector3 a, struct Vector3 b) {  
    ...  
}
```

Но...

? Откуда этот файл знает,
что такое `struct Vector3`?

Нам чего-то не хватает

Оба файла используют одну и ту же информацию.

```
main.c
```

```
    ?
```

```
vector3.c
```

Где хранить описание структуры?

Где хранить объявления функций?

Появляется заголовочный файл

main.c



vector3.h



vector3.c

Оба файла подключают один общий заголовок.

Что находится в vector3.h?

```
struct Vector3 {  
    double x;  
    double y;  
    double z;  
};  
  
struct Vector3 add(struct Vector3 a, struct Vector3 b);  
  
double length(struct Vector3 v);  
  
void printVector(struct Vector3 v);
```

Есть ли здесь вычисления?

```
struct Vector3 add(struct Vector3 a, struct Vector3 b );
```

```
double length(struct Vector3 v);
```

```
void printVector(struct Vector3 v);
```

✗ Нет.

В заголовочном файле нет реализации функций.

Что хранится в заголовочном файле?

Обычно содержит

- описание типов;
- объявления функций;
- КОНСТАНТЫ.

Не содержит

- код функций;
- вычисления;
- реализацию алгоритмов.

Если в `.h` нет кода...

Тогда где находится реализация функций?

Ответ — в исходном файле.

`vector3.c`

↓

код функций

Что находится в `vector3.c` ?

```
#include "vector3.h"

struct Vector3 add(struct Vector3 a, struct Vector3 b) {
    struct Vector3 result = { a.x + b.x, a.y + b.y, a.z + b.z };

    return result;
}

double length(struct Vector3 v) {
    return sqrt(v.x * v.x + v.y * v.y + v.z * v.z);
}
```

Объявление и определение

Объявление

```
double length(struct Vector3 v);
```

Сообщает компилятору,
что функция существует.

Определение

```
double length(struct Vector3 v) {  
    return sqrt(v.x * v.x + v.y * v.y + v.z * v.z);  
}
```

Содержит реализацию функции.

Интерфейс и реализация

Интерфейс

Что умеет модуль.

```
vector3.h
```

Реализация

Как модуль это делает.

```
vector3.c
```

Итоговая структура проекта

```
project/
```

```
|— main.c  
|— vector3.h  
└— vector3.c
```

main.c

Использует модуль.

vector3.h

Описывает модуль.

vector3.c

Реализует модуль.

Новый вопрос

Хорошо.

Но как содержимое

```
vector3.h
```

попадает в

```
main.c
```

?

Подключение заголовочного файла

```
#include "vector3.h"

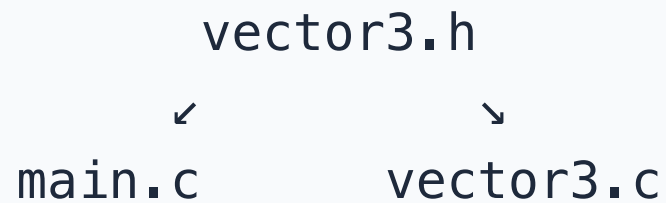
int main(void) {
    struct Vector3 a = {1.0, 2.0, 3.0};
    struct Vector3 b = {4.0, 5.0, 6.0};

    struct Vector3 c = add(a, b);

    printVector(c);
}
```

`#include` подключает описание модуля.

Заголовок подключают оба файла



- `main.c` узнаёт о типе `Vector3` и функциях.
- `vector3.c` использует те же объявления при реализации.

Почему это удобно?

Представим, что сигнатура функции изменилась.

Было:

```
double length(struct Vector3 v);
```

Стало:

```
float length(struct Vector3 v);
```

Изменение выполняется **в одном месте** — в заголовочном файле.

Все файлы используют одно и то же описание.

Пока мы знаем только это

main.c

↓

```
#include "vector3.h"
```

vector3.c

↓

```
#include "vector3.h"
```

Этого достаточно, чтобы пользоваться модулем.

Как именно работает `#include`, мы рассмотрим позже.

Один файл уже недостаточно

Раньше программа состояла из одного файла.

```
gcc main.c
```

Теперь файлов несколько.

```
main.c
```

```
vector3.c
```

? Как собрать одну программу из нескольких файлов?

Компилируем оба файла

Компилятору нужно передать все исходные файлы проекта.

```
gcc main.c vector3.c
```

Каждый файл компилируется отдельно.

Что делает компилятор?

Компилятор переводит исходный код
в объектный файл.

`main.c`

↓

`main.o`

`vector3.c`

↓

`vector3.o`

Каждый исходный файл
компилируется независимо.

Почему это возможно?

Во время компиляции файла

```
main.c
```

компилятор знает только

- содержимое `main.c`;
- подключённые заголовочные файлы.

Он не читает `vector3.c`.

После компиляции

Получаются два объектных файла.

```
main.o
```

```
vector3.o
```

Но исполняемой программы
ещё нет.

Кто собирает программу?

После компиляции запускается

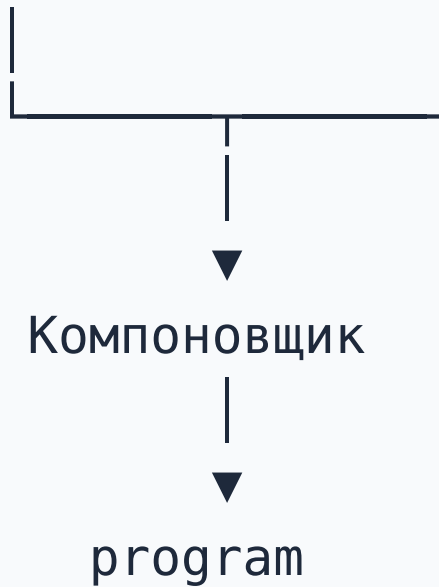
Компоновщик

(англ. **linker**)

Его задача —
объединить объектные файлы
в одну программу.

Работа компоновщика

main.o vector3.o



Компоновщик соединяет
все части программы.

Что делает компоновщик?

Во время компиляции

```
struct Vector3 c = add(a, b);
```

компилятор знает,
что функция существует.

Но её код находится
в другом объектном файле.

Компоновщик находит реализацию
и соединяет её с вызовом.

Что будет, если забыть один файл?

Например,

```
gcc main.c
```

Компилятор успешно обрабатывает

`main.c`.

Но...

Ошибка компоновки

Компоновщик не сможет найти

```
add(...)
```

и сообщит об ошибке.

```
undefined reference to 'add'
```

Когда возникает ошибка?

Во время компиляции

Нет объявления.

Например,

```
add(a, b);
```

но компилятор

не знает,

что такое `add()`.

Во время компоновки

Есть объявление,

но отсутствует определение.

Например,

не подключили

```
vector3.c
```

Как теперь выглядит сборка программы?



Итоги

`main.c`



Использует модуль

`vector3.h`



Описывает модуль

`vector3.c`



Реализует модуль

Компилятор



Компилирует каждый файл отдельно

Компоновщик



Объединяет программу

Что происходит перед компиляцией?

На прошлой лекции мы говорили,
что программа компилируется так.

```
main.c
```

```
↓
```

```
Компилятор
```

```
↓
```

```
main.o
```

Но это не вся картина

Перед компилятором работает ещё одна программа.

```
main.c
```

```
↓
```

```
Препроцессор
```

```
↓
```

```
Компилятор
```

```
↓
```

```
main.o
```

Что делает препроцессор?

Препроцессор изменяет исходный текст программы.

Например,

- подключает другие файлы;
- заменяет один текст другим;
- удаляет ненужные части программы.

Только после этого запускается компилятор.

Первая директива препроцессора

Мы уже использовали

```
#include "vector3.h"
```

Но что делает эта строка?

До работы препроцессора

```
#include "vector3.h"

int main(void) {
    struct Vector3 a = {1, 2, 3};
    printVector(a);
    return 0;
}
```

После работы препроцессора

```
struct Vector3 {  
    double x;  
    double y;  
    double z;  
};  
  
void printVector(struct Vector3);  
  
int main(void)  
{  
    struct Vector3 a = {1, 2, 3};  
    printVector(a);  
    return 0;  
}
```

Что произошло?

Содержимое файла

```
vector3.h
```

было **вставлено** вместо строки

```
#include "vector3.h"
```

Важно

`#include`

не подключает библиотеку.

не компилирует файл.

не выполняет программу.

Он только вставляет текст одного файла в другой.

Можно подключать несколько файлов

```
#include <stdio.h>
#include <math.h>

#include "vector3.h"
#include "matrix3.h"

int main(void)
{
    ...
}
```

Именно этот файл увидит компилятор

Исходные файлы

↓

Препроцессор

↓

Один большой текст

↓

Компилятор

Компилятор ничего не знает о директиве `#include`.

Он получает уже готовый текст программы.

Возникает новая проблема

Представим такой проект.

```
main.c
```

```
|— vector3.h
```

```
|— matrix3.h
```

```
    |  
    |— vector3.h
```

Что произойдёт после работы препроцессора?

После препроцессора

Получится примерно такой текст.

```
struct Vector3
{
    ...
};

...

struct Vector3
{
    ...
};
```

Результат

Компилятор сообщит об ошибке.

```
redefinition of 'struct Vector3'
```

или

```
duplicate definition
```

Как избежать повторного включения одного и того же файла?

Как избежать повторного включения?

Нужно сказать препроцессору:

Если этот файл уже был подключён,
больше не вставляй его.

Для этого используются
условные директивы препроцессора.

Первая условная директива

```
#ifndef VECTOR3_H
```

Читается как

| If Not Defined

или

| Если имя ещё не определено

Полный пример

```
#ifndef VECTOR3_H
#define VECTOR3_H

struct Vector3
{
    double x;
    double y;
    double z;
};

void printVector(struct Vector3);

#endif
```

Как это работает?

При первом подключении файла

```
#ifndef VECTOR3_H
```

Имя `VECTOR3_H`
ещё не существует.



Условие истинно.

Следующая строка

```
#define VECTOR3_H
```

Препроцессор создаёт имя

```
VECTOR3_H
```

Теперь он "помнит",
что этот файл уже был обработан.

Затем

Выполняется всё содержимое файла.

```
struct Vector3
{
    ...
};

void printVector(...);

...
```

И наконец

```
#endif
```

Заканчивает условный блок.

На этом первое подключение завершено.

Что произойдёт во второй раз?

Препроцессор снова встретит

```
#ifndef VECTOR3_H
```

Но теперь имя

```
VECTOR3_H
```

уже существует.

Поэтому

Условие становится ложным.

Весь файл пропускается.

```
#ifndef VECTOR3_H  
  
    ...  
  
#endif
```



Ничего не вставляется.

Что получилось?

Первое подключение

```
vector3.h
```



Файл вставляется.

Повторное подключение

```
vector3.h
```



Файл пропускается.

Include Guard

```
#ifndef VECTOR3_H  
#define VECTOR3_H  
  
...  
  
#endif
```

Позволяет безопасно подключать
заголовочный файл
сколько угодно раз.

Современная альтернатива

Во многих проектах используют

```
#pragma once
```

Он означает

Подключить этот файл
только один раз.

Что выбрать?

Include Guard

```
#ifndef  
#define  
#endif
```

Работает практически
во всех компиляторах.

pragma once

```
#pragma once
```

Короче и проще.

Поддерживается
почти всеми современными
компиляторами.

До сих пор мы использовали

```
#define VECTOR3_H
```

Но что вообще делает

```
#define
```

?

Что ещё умеет `#define` ?

До сих пор мы использовали

```
#define VECTOR3_H
```

Но `#define`

МОЖНО ИСПОЛЬЗОВАТЬ

не только в Include Guard.

Самый простой пример

```
#define PI 3.141592653589793
```

Что произойдёт дальше?

До работы препроцессора

```
#define PI 3.141592653589793
```

```
double length = 2 * PI * r;
```

После работы препроцессора

```
double length = 2 * 3.141592653589793 * r;
```

Имя **PI**

было заменено
на другой текст.

Что сделал препроцессор?

-  Не вычислил выражение.
-  Не выполнил программу.
-  Не понял язык C.
-  Просто заменил один текст другим.

Ещё один пример

```
#define BUFFER_SIZE 1024  
  
char buffer[BUFFER_SIZE];
```



После препроцессора

```
char buffer[1024];
```

Важно

Препроцессор

не знает, что такое

- переменная;
- функция;
- тип;
- выражение.

Для него программа —
это обычный текст.

Можно ли заменять не только имя?

Да.

`#define`

может принимать параметры.

```
#define SQUARE(x) x * x
```

Что получится?

```
SQUARE(a + b)
```

Попробуйте представить,
какой текст получит компилятор.

После работы препроцессора

```
a + b * a + b
```

Совсем не тот результат,
который ожидался.

Почему так произошло?

Препроцессор

не знает правил арифметики.

Он просто выполнил замену текста.

Обычно пишут так

```
#define SQUARE(x) ((x) * (x))
```

Скобки помогают
избежать подобных ошибок.

Но появляется новая проблема

Что получится?

```
SQUARE(i++)
```

После препроцессора

```
((i++) * (i++))
```

Аргумент был подставлен
два раза.

Значит,

`i` увеличится дважды.

Вывод

Макросы

- ✓ удобны для простых замен текста;
- ⚠ требуют осторожности;
- ✓ во многих случаях обычная функция будет безопаснее и понятнее.

Представим

```
#define VALUE int  
  
VALUE x = 5;
```



После препроцессора

```
int x = 5;
```

Для препроцессора `VALUE` — это просто последовательность символов.

Он не знает, что получилось имя типа.

```
#define CAT dog  
  
char *CAT = "CAT";
```



После препроцессора

```
char *dog = "CAT";
```

Заменяются только идентификаторы.

Строковые литералы
не изменяются.

Одна программа — разные версии

Иногда одну и ту же программу
нужно собирать по-разному.

Например,

- для Windows;
- для Linux;
- для режима отладки;
- для финальной версии.

Как это сделать?

Можно ли сказать препроцессору

| Оставь только эту часть программы,
а остальной код удалить
ещё до компиляции?

Директива `#ifdef`

```
#ifdef DEBUG
```

```
printf("x = %d\n", x);
```

```
#endif
```

`#ifdef`

означает

Если имя определено...

Как это работает?

```
#ifdef DEBUG
```

Если существует имя `DEBUG` то код между

```
#ifdef
```

и

```
#endif
```

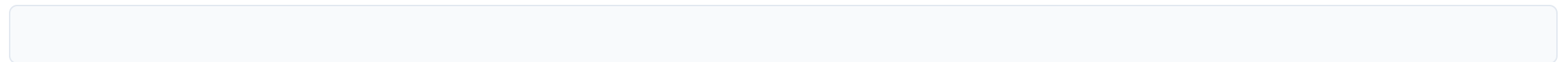
останется в программе.

Если имя не определено

```
#ifdef DEBUG  
  
printf("x = %d\n", x);  
  
#endif
```



После препроцессора



Весь блок будет удалён.

Если имя определено

До препроцессора

```
#ifdef DEBUG
```

```
printf("x = %d\n", x);
```

```
#endif
```

```
foo();
```

После препроцессора

```
printf("x = %d\n", x);  
  
foo();
```

Код внутри блока останется в программе.

Можно выбрать одну из двух версий

```
#ifdef DEBUG  
  
printf("Debug mode\n");  
  
#else  
  
printf("Release mode\n");  
  
#endif
```

Пример

```
#ifdef _WIN32

printf("Windows\n");

#else

printf("Linux или macOS\n");

#endif
```

Во время компиляции останется только одна ветка.

Ещё один пример

```
#ifdef DEBUG  
  
printf("x = %d\n", x);  
  
printf("y = %d\n", y);  
  
#endif
```

Во время разработки можно выводить дополнительную информацию.

В финальной версии этот код исчезнет.

Директива `#undef`

Можно удалить имя, созданное ранее.

```
#define SIZE 100
```

```
...
```

```
#undef SIZE
```

После этого `SIZE` снова считается неопределённым.

Что увидит компилятор?

Можно попросить компилятор
показать результат работы препроцессора.

```
gcc -E main.c
```

Что получится?

Будет создан новый
исходный текст программы.

В нём уже не останется

- `#include ;`
- `#define ;`
- `#ifdef ;`
- `#ifndef .`

Компилятор работает
именно с ЭТИМ текстом.

Полный путь программы

Исходные файлы

↓

Препроцессор

↓

Новый исходный файл

↓

Компилятор

↓

Объектный файл (.o)

↓

Компоновщик

↓

Исполняемый файл

Итоги

Сегодня мы познакомились с

- `#include` — вставка текста;
- `#ifndef` и Include Guard — защита от повторного включения;
- `#pragma once` — современная альтернатива;
- `#define` — текстовая замена;
- макросами с параметрами;
- `#ifdef` — условная компиляция;
- `#undef` — удаление имени.

Все эти директивы

выполняются **до компиляции**.

Несколько файлов — одна программа

Мы научились

разделять программу
на несколько файлов.

Но возникает вопрос.

Как файлы
обмениваются данными?

Пример

```
main.c
```

```
printf("%d\n", counter);
```



Ошибка.

Почему?

Где находится `counter` ?

```
// globals.c
```

```
int counter = 0;
```

```
// main.c
```

```
printf("%d\n", counter);
```

Компилятор не знает, что переменная существует в другом файле.

Директива `extern`

Можно сообщить компилятору,
что объект существует
в другом месте.

```
extern int counter;
```

Что означает `extern` ?

```
extern int counter;
```

Создаётся

☐ нет

Выделяется память

☐ нет

Сообщается компилятору

☒ да

Определение и объявление

```
// globals.c
```

```
int counter = 0;
```

```
// main.c
```

```
extern int counter;
```

Определение создаёт объект.

Объявление только сообщает о нём.

В программе должно быть

Определение



ровно одно

Объявлений



сколько угодно

А если наоборот?

Иногда переменная не должна быть доступна из других файлов.

Как это сделать?

Ключевое слово `static`

```
// globals.c  
  
static int counter = 0;
```

Теперь `counter` видна только внутри этого файла.

Что произойдёт?

```
// main.c  
  
extern int counter;
```



Ошибка.

Переменная скрыта от остальных файлов.

static на уровне файла

```
main.c
```

```
counter
```

×

```
globals.c
```

```
counter
```

Это разные объекты.

Их имена не конфликтуют.

Сравнение

	extern	static
Виден из других файлов	✓	✗
Создаёт объект	✗	✓

Но есть ещё одна роль `static`

```
void foo()  
{  
    static int count = 0;  
  
    count++;  
  
    printf("%d\n", count);  
}
```

Что выведет программа?

Первый вызов

1

Второй

2

Третий

3

Почему?

Что произошло?

Обычная локальная переменная создаётся при каждом вызове функции.

`static` создаётся только один раз.

Сравнение

```
void foo() {  
    int x = 0;  
}
```

создаётся при каждом вызове.

```
void foo()  
{  
    static int x = 0;  
}
```

создаётся один раз и сохраняется между вызовами.

Две роли `static`

```
static int counter;
```

скрывает объект от других файлов.

```
void foo()  
{  
    static int counter;  
}
```

сохраняет значение между вызовами функции.

Ещё одна проблема

```
while (flag == 0)
{
}
```

В другом месте программы

```
flag = 1;
```

Почему цикл может никогда не закончиться?

Причина

Компилятор может решить, что значение `flag` не изменяется. И сохранить его в регистре процессора.

Ключевое слово **volatile**

```
volatile int flag;
```

Теперь каждое чтение и каждая запись будут выполняться через память.

Когда используют `volatile` ?

Редко.

Например,

- устройства ввода-вывода;
- обработчики прерываний;
- разделяемые данные.

Для синхронизации потоков одного `volatile` недостаточно.

Итоги

Сегодня мы познакомились с

- `extern` — объявление объекта в другом файле;
- `static` — ограничение области видимости;
- `static` внутри функции — сохранение значения между вызовами;
- `volatile` — запрет некоторых оптимизаций доступа к памяти.